Programmable Stepper Motor Sequence Controller using PIC16F877A

28BYJ-48 + ULN2003A | SSD1306 OLED | 4x4 Keypad | 20 MHz | XC8 | Sequence Runner v6

Team Members

Roll No	Name
CB.EN.U4ELC24109	HARIE GOUTAYM
CB.EN.U4ELC24156	V R BHIRUGUDEV
CB.EN.U4ELC24159	SHRREYAS VIJAY
CB.EN.U4ELC24160	NAVIN RUBAG V P

Introduction

This project implements a full-featured programmable stepper motor sequence controller on the PIC16F877A microcontroller operating at 20 MHz. The system allows the user to compose a sequence of up to 5 independent motor movement steps, configure each step's type, direction (CW or CCW), and speed (1–15 RPM), and then execute the complete sequence a user-specified number of times (1–99). All user interaction is handled through a 4x4 membrane keypad and a 128x64 SSD1306 OLED display driven over software bit-bang I2C.

Version 6 of the firmware introduces a cleaner architecture: the INT (Intermittent) step type (now labelled Intermittent / INT) takes explicit spin seconds, idle seconds, and a cycle count, and executes that exact number of ON/IDLE cycles automatically without requiring the user to press * to stop. The run mode asks for a repeat count (1–99) before execution begins, iterates that many times, then auto-saves the sequence and returns to the menu. The I2C driver now uses proper open-drain toggling for improved bus compatibility. The motor is driven using an 8-step

half-stepping pattern through the ULN2003A Darlington driver board, producing smooth motion at 4096 half-steps per revolution.

Objectives

- Compose and store up to 5 stepper motor steps of four types: Angle, Rotations, Intermittent (INT), and Timed.
- Configure direction (CW/CCW) and speed (1–15 RPM) independently for every step.
- Execute the full sequence a user-specified number of times (1–99 repeats), with the iteration counter shown live on the OLED.
- Display all menus, input prompts, step details, and live countdowns on the SSD1306 OLED.
- Drive the 28BYJ-48 motor using Timer1 interrupt-driven 8-step half-stepping for smooth, accurate motion.
- Implement non-blocking keypad scanning for real-time stop control during motor runs.
- Auto-save the sequence after every successful run; reload it at any time via key 6.

Components Used

Component	Role
PIC16F877A	Main microcontroller @ 20 MHz crystal
28BYJ-48	5V unipolar stepper motor, 4096 half-steps/rev
ULN2003A	Darlington driver board for all four motor coils
SSD1306 OLED	128x64 I2C display for menus and live status
4x4 Keypad	User input: numbers, direction, confirm, stop
5V DC Supply	Powers PIC and logic; separate 5V rail for motor

Block Diagram

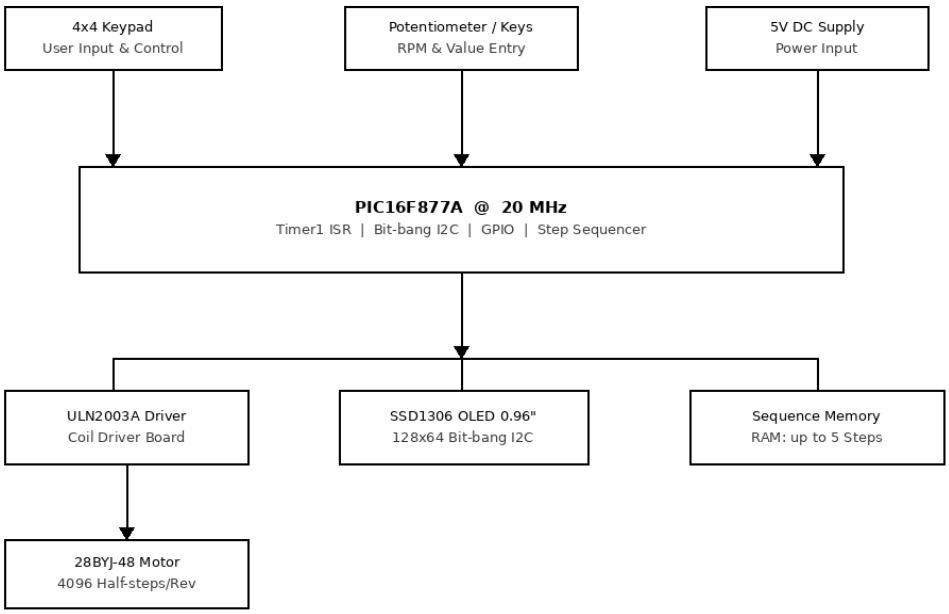


Figure 1. System Block Diagram

Hardware and Pin Connections

Port Pin	Phys. Pin	Direction	Connected To	Function
RD0	Pin 19	Output	ULN2003A IN1	Motor coil 1
RD1	Pin 20	Output	ULN2003A IN2	Motor coil 2
RD2	Pin 21	Output	ULN2003A IN3	Motor coil 3
RD3	Pin 22	Output	ULN2003A IN4	Motor coil 4
RB0–RB3	Pin 33–36	Input	Keypad COL1–4	Keypad columns (pull-up ON)
RB4–RB7	Pin 37–40	Output	Keypad ROW1–4	Keypad rows (driven LOW one at a time)
RC3	Pin 18	Output	OLED SCL	Bit-bang I2C clock
RC4	Pin 23	Bidir.	OLED SDA	Bit-bang I2C data / ACK
VDD	Pin 11/32	Power	5V Supply	Microcontroller power
VSS	Pin 12/31	Ground	GND	Microcontroller ground

Keypad Layout and Key Functions

Key	Label	Function
1	ADD ANGLE	Add a step: rotate motor by 1–3600 degrees
2	ADD ROT	Add a step: rotate 1–99 full turns
3	ADD INT	Add an Intermittent step: spin for N sec, idle for M sec (0=no pause), repeat C times automatically
4	ADD TIMED	Add a step: run motor for 1–99 seconds with live OLED countdown
5	VIEW	Review all programmed steps on the OLED sequence list displayed on the main menu
6	LOAD	Reload the last saved sequence from RAM buffer
#	RUN / OK	Run the full sequence OR confirm a number entry
*	STOP / BACK	Emergency stop, backspace during input, or cancel
0–9	Numbers	Enter numeric values during input prompts

Working Principle

Step Types

The controller supports four step types. Each is programmed by pressing the corresponding key on the main menu (1=Angle, 2=Rotations, 3=Intermittent, 4=Timed), entering the required value(s), selecting direction, and confirming RPM. All numeric input uses the # key to confirm and * to backspace or cancel.

Type	Key	Description
ANGLE	1	Rotate by precise degrees (1–3600). steps = (degrees x 4096) / 360
ROTATIONS	2	Rotate 1–99 full turns. steps = rotations x 4096
CYCLE	3	Motor ON for N sec, IDLE for M sec (0=no pause), repeated C times (1–99). Fully automatic — no * required
TIMED	4	Run motor for 1–99 seconds with live countdown on OLED. Press * to abort early

Angle Calculation

For ANGLE steps the firmware converts degrees to half-steps using integer arithmetic:

```
half-steps = (degrees x 4096) / 360
```

Example: 180 degrees = 2048 half-steps = one half-revolution. For ROTATION steps: half-steps = rotations x 4096.

Run Modes

Pressing # on the main menu prompts the user to enter a repeat count (1–99) before execution begins. There is no separate mode selection — count-based execution is the only run mode in v6.

Run Mode	Behaviour
COUNT	Enter 1–99 repeats. OLED shows current iteration number live. After all iterations complete, the sequence is auto-saved and the firmware returns to the menu.

Sequence Execution Flow

1. User enters repeat count (1–99) and execution begins immediately.
2. Steps execute in order from step 1 to step N, each displaying type, direction, and RPM on the OLED.
3. For TIMED steps the OLED shows a live second countdown. For INT (Intermittent) steps, the ON phase, IDLE phase, and cycle counter are all managed automatically.
4. A 300 ms pause separates consecutive steps. Pressing * aborts the sequence immediately at any time.
5. On completion, the sequence is automatically saved to the saved[] RAM buffer (no user prompt). The done screen offers # to reload and run again, or * to clear.

Saving and Loading

After every successful sequence run, the firmware automatically calls autosave(), copying the current step array (seq[]) into the saved[] RAM buffer. The user is not prompted — saving is always performed. Pressing key 6 on the main menu calls autoload(), restoring the saved sequence into seq[].er (sv[]) in RAM. Pressing 6 on the main menu restores sv[] back into sq[], making the saved sequence ready to run again. Saved steps are lost on power-off as EEPROM write is not implemented in v3.

OLED Screen Guide

Feature	Detail
Resolution	128 x 64 pixels, 8 pages of 8 pixels each
Characters/row	21 characters (128 / 6 pixels per char)
Font	5x8 bitmap, extended set including V, Y, I, M, N, P, Q, /, :
I2C Address	0x3C (7-bit) / 0x78 (8-bit with R/W bit)
Charge pump	Enabled in software (0x8D, 0x14) — no external capacitor
Inverted text	Title bars rendered with bitwise NOT for highlight effect

All title bars use inverted rendering (white text on black background) achieved by bitwise-NOT of each font byte. The SSD1306 initialisation sequence enables the internal charge pump, sets page addressing mode, and configures correct column and row scanning orientation for the standard module layout.

Firmware Architecture

Timer1 – Interrupt-Driven Step Engine

All motor motion is managed entirely within the Timer1 ISR. The main loop never waits in a busy loop for the motor — it only monitors the step counter (vl) while polling the keypad for an abort press. This architecture ensures that * can stop motion at any time without blocking the OLED or keypad.

Parameter	Value
Timer module	Timer1 (16-bit)
Prescaler	1:8 (T1CON = 0x30)

Mode	Interrupt on overflow (PIE1bits.TMR1IE = 1)
Reload formula	reload = 65536 - (37500000 / (4096 x RPM))
RPM range	1 to 15 RPM
ISR action	Write HS[vi] to PORTD, advance vi (CW) or decrement (CCW), decrement vl; de-energise when vl = 0

8-Step Half-Step Coil Pattern

The 28BYJ-48 is driven with an 8-step half-stepping pattern stored in the HS[] array. CW rotation advances the index (0→1→...→7→0). CCW rotation decrements it (7→6→...→0→7). This produces 4096 steps per revolution and smoother torque compared to full-stepping.

Step	IN1 (RD0)	IN2 (RD1)	IN3 (RD2)	IN4 (RD3)	Hex
0	1	0	0	1	0x09
1	1	0	0	0	0x01
2	1	1	0	0	0x03
3	0	1	0	0	0x02
4	0	1	1	0	0x06
5	0	0	1	0	0x04
6	0	0	1	1	0x0C
7	0	0	0	1	0x08

Software Bit-bang I2C

The SSD1306 OLED is driven by manually toggling RC3 (SCL) and RC4 (SDA) with 3 µs delays per edge using open-drain macros (SH/SL/DH/DL) that release the TRIS bit to go HIGH rather than actively driving it. This is correct I2C open-drain behaviour. After each byte the firmware releases SDA (TRISC4=1) and checks RC4 for the ACK pulse before driving SCL low again.irmware switches RC4 to input mode to read the ACK bit with a 200-cycle timeout to prevent bus hang. The hardware MSSP module is not used, keeping it free for other peripherals.

Keypad Scanning

The ks() function drives each PORTB row pin (RB4–RB7) LOW in turn and reads columns (RB0–RB3) with PORTB pull-ups enabled. A 15 ms software debounce is applied on both press and release. ks() returns 0 immediately if no key is held, allowing non-blocking calls from within run_step() to detect an emergency stop during motor execution.

Build Configuration

Configuration Bits

Bit	Setting	Reason
FOSC	HS	High-speed crystal at 20 MHz
WDTE	OFF	Watchdog timer disabled
PWRTE	ON	Power-up timer for supply stability
BOREN	ON	Brown-out reset enabled
LVP	OFF	Low-voltage programming disabled
CP / CPD / WRT	OFF	Code and data protection off

Build Steps

6. Open MPLAB X IDE and ensure the XC8 compiler is installed (free version sufficient).
7. Right-click project → Properties → XC8 Compiler → set Optimisation Level to 1 (pragma optimization_level 1 is also set in source).
8. Click Clean and Build. Verify that program space used is within 1E00h words.
9. Program using PICkit 3/4 or TinyMultiBootLoader+ via UART at 115200 baud.
10. For bootloader upload: power-cycle the PIC, then click Write. Wait for Verify OK.

Troubleshooting

Problem	Likely Cause	Fix
OLED blank	Wrong I2C address / wiring	Check RC3/RC4 connections; ensure 3.3V or 5V to OLED VCC
OLED garbled	Wrong _XTAL_FREQ	Verify 20 MHz crystal; set _XTAL_FREQ = 20000000UL

Motor not moving	No external 5V to motor	Connect external 5V to ULN2003A VCC — do NOT use PIC pin
Motor vibrates, no rotation	Wrong coil sequence	Verify HS[] array matches your specific motor wiring order
Keypad not responding	Pull-ups not enabled	Check OPTION_REGbits.nRBPU = 0 in kinit()
Steps lost on power off	RAM-only storage	Normal behaviour — EEPROM save not implemented in v6. Use key 6 to reload last run's auto-saved sequence.
Sequence won't run	No steps added	Add at least one step with keys 1–4 before pressing #
HEX too large	Optimisation not set	Set XC8 optimisation level to 2 in project properties

Technical Specifications

Parameter	Value
Microcontroller	PIC16F877A @ 20 MHz external crystal
Flash memory	8 KB (7680 words usable with bootloader)
RAM	368 bytes
Motor	28BYJ-48 unipolar stepper
Steps per revolution	4096 half-steps
Speed range	1 to 15 RPM
Max sequence steps	5 steps per sequence
Step types	Angle, Rotations, Cycle, Timed
Run modes	Count (1–99 repeats)
Display	SSD1306 0.96" OLED 128x64, bit-bang I2C @ 0x78
Input	4x4 membrane keypad with 15 ms debounce
Operating voltage	5V DC (PIC/logic); separate 5V for motor

Compiler	XC8 Free, Optimisation Level 1

Full Source Code

Complete firmware for Sequence Runner v6. Compiler: XC8 Free. Target: PIC16F877A @ 20 MHz.

```
/*
 * PIC16F877A Stepper Sequence Runner v6
 * 20MHz | 28BYJ-48 | SSD1306 I2C | 4x4 Keypad
 * RD0-RD3=Coils RB0-RB3=KCols RB4-RB7=KRows RC3=SCL RC4=SDA
 * * = abort run, # = run, 1-4 = add step, 6 = load saved
 */
#include <xc.h>
#include <stdint.h>
#pragma optimization level 1

#pragma config FOSC=HS,WDTE=OFF,PWRTE=ON,BOREN=ON,LVP=OFF,CP=OFF,CPD=OFF,WRT=OFF
#define _XTAL_FREQ 20000000UL
#define SPR        4096U
#define OA         0x78
#define MAXS       5

/* forward decls */
static char ks(void);
static char kw(void);

/* ======= SEQUENCE DATA ======= */
typedef struct{uint8_t type;uint32_t val;uint8_t val2;uint8_t cyc;uint8_t cw;uint8_t
rpm;}Step;
static Step seq[MAXS], saved[MAXS];
static uint8_t sc=0, savedc=0;

/* ======= I2C ======= */
#define SH  TRISCbits.TRISC3=1
#define SL  TRISCbits.TRISC3=0;PORTCbits.RC3=0
#define DH  TRISCbits.TRISC4=1
#define DL  TRISCbits.TRISC4=0;PORTCbits.RC4=0
#define DY  __delay_us(3)

static void i2s(void){DH;DY;SH;DY;DL;DY;SL;DY;}
static void i2p(void){DL;DY;SH;DY;DH;DY;}
static void i2b(uint8_t d){
    uint8_t i;
    for(i=0;i<8;i++){if(d&0x80){DH;}else{DL;}DY;SH;DY;SL;DY;d<<=1;}
    DH;DY;SH;DY;{uint8_t t=100;while(PORTCbits.RC4&&--t);}SL;DY;
}
static void ow(uint8_t dc,uint8_t v){i2s();i2b(OA);i2b(dc);i2b(v);i2p();}
#define OC(c) ow(0x00,c)

/* ======= OLED FONT ======= */
static const uint8_t F[][5]={
{0x3E,0x51,0x49,0x45,0x3E},{0x00,0x42,0x7F,0x40,0x00}, /* 0  1  */
{0x42,0x61,0x51,0x49,0x46},{0x21,0x41,0x45,0x4B,0x31}, /* 2  3  */
{0x18,0x14,0x12,0x7F,0x10},{0x27,0x45,0x45,0x45,0x39}, /* 4  5  */
{0x3C,0x4A,0x49,0x49,0x30},{0x01,0x71,0x09,0x05,0x03}, /* 6  7  */
```

```
{0x36,0x49,0x49,0x49,0x36},{0x06,0x49,0x49,0x29,0x1E}, /* 8  9  */
{0x7E,0x11,0x11,0x11,0x7E},{0x7F,0x09,0x09,0x09,0x06}, /* A=10 B=11 */
{0x3E,0x41,0x41,0x41,0x22},{0x7F,0x41,0x41,0x22,0x1C}, /* C=12 D=13 */
{0x7F,0x49,0x49,0x49,0x41},{0x3E,0x41,0x49,0x49,0x7A}, /* E=14 G=15 */
{0x00,0x41,0x7F,0x41,0x00},{0x38,0x04,0x02,0x04,0x38}, /* I=16 M=17 */
{0x7F,0x04,0x08,0x10,0x7F},{0x3E,0x41,0x41,0x41,0x3E}, /* N=18 O=19 */
{0x7F,0x09,0x19,0x29,0x46},{0x46,0x49,0x49,0x49,0x31}, /* R=20 S=21 */
{0x01,0x01,0x7F,0x01,0x01},{0x3F,0x40,0x40,0x40,0x3F}, /* T=22 U=23 */
{0x1F,0x20,0x40,0x20,0x1F},{0x63,0x14,0x08,0x14,0x63}, /* V=24 Y=25 */
{0x00,0x00,0x00,0x00,0x00},{0x08,0x08,0x08,0x08,0x08}, /* sp=26 -=27 */
{0x7F,0x40,0x40,0x40,0x40},{0x3F,0x40,0x38,0x40,0x3F}, /* L=28 W=29 */
{0x7F,0x09,0x09,0x09,0x06},{0x63,0x55,0x49,0x41,0x41}, /* P=30 Q=31 */
{0x20,0x10,0x08,0x04,0x02},{0x00,0x36,0x36,0x00,0x00}, /* /=32 :=33 */
};
static uint8_t fi(char c){
    if(c>='0'&&c<='9')return c-'0';
    switch(c){
        case 'A':return 10;case 'B':return 11;case 'C':return 12;
        case 'D':return 13;case 'E':return 14;case 'G':return 15;
        case 'I':return 16;case 'M':return 17;case 'N':return 18;
        case 'O':return 19;case 'R':return 20;case 'S':return 21;
        case 'T':return 22;case 'U':return 23;case 'V':return 24;
        case 'Y':return 25;case '-':return 27;case 'L':return 28;
        case 'W':return 29;case 'P':return 30;
        case '/':return 32;case ':':return 33;
    }
    return 26; /* space */
}
static void ogoto(uint8_t c,uint8_t r){OC(0xB0|r);OC(c&0xF);OC(0x10|(c>>4));}
static void och(uint8_t c,uint8_t r,char ch,uint8_t inv){
    uint8_t i,x=fi(ch);
    ogoto(c,r);i2s();i2b(OA);i2b(0x40);
    for(i=0;i<5;i++){uint8_t b=F[x][i];i2b(inv?~b:b);}
    i2b(inv?0xFF:0);i2p();
}
static void ost(uint8_t c,uint8_t r,const char*s,uint8_t inv){
    while(*s&&c+6<=128){och(c,r,*s++,inv);c+=6;}
}
static void onum(uint8_t c,uint8_t r,uint32_t v,uint8_t w){
    char b[6];uint8_t i=w;b[w]=0;
    do{b[--i]='0'+v%10;v/=10;}while(v&&i);
    while(i)b[--i]=' ';
    ost(c,r,b,0);
}
static void oclr_row(uint8_t r){
    uint8_t i;ogoto(0,r);
    i2s();i2b(OA);i2b(0x40);
    for(i=0;i<128;i++)i2b(0);
    i2p();
}
static void oclr(void){uint8_t r;for(r=0;r<8;r++)oclr_row(r);}
static void oinit(void){
    __delay_ms(100);
    OC(0xAE);OC(0xD5);OC(0x80);OC(0xA8);OC(0x3F);
    OC(0xD3);OC(0x00);OC(0x40);OC(0x8D);OC(0x14);
    OC(0x20);OC(0x02);OC(0xA1);OC(0xC8);
    OC(0xDA);OC(0x12);OC(0x81);OC(0xCF);
    OC(0xD9);OC(0xF1);OC(0xDB);OC(0x40);
    OC(0xA4);OC(0xA6);OC(0xAF);
    oclr();
}
static void otit(const char*s){
    uint8_t i;ogoto(0,0);
    i2s();i2b(OA);i2b(0x40);
```

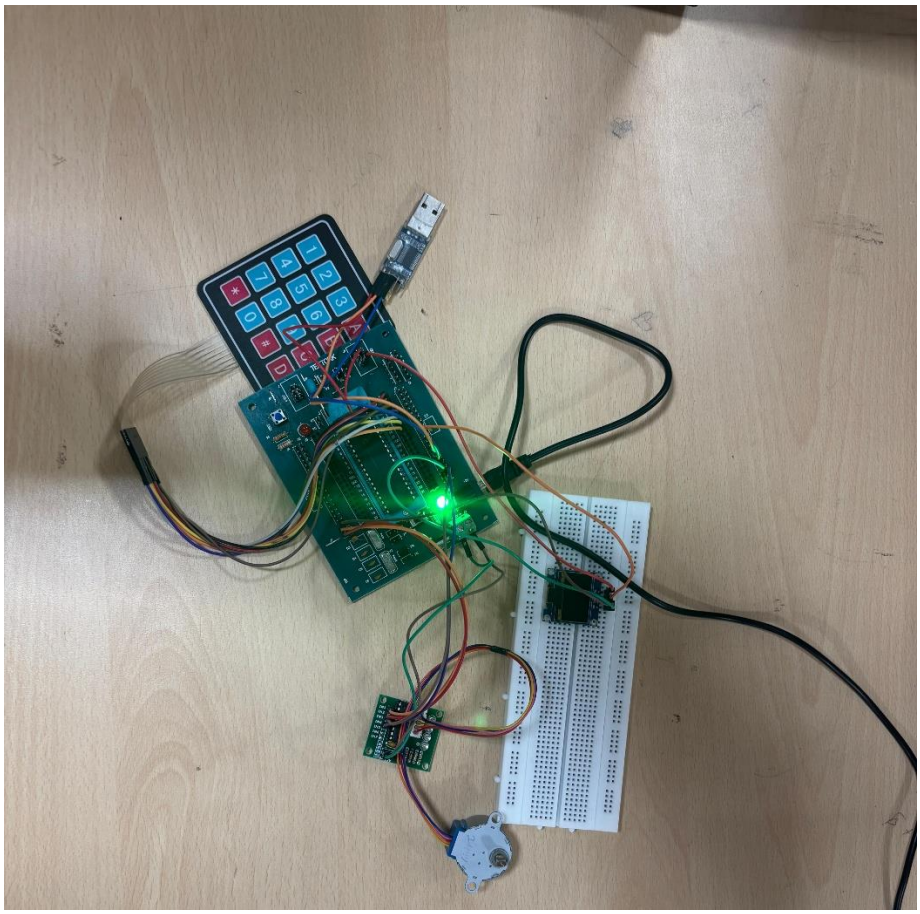
```
    for(i=0;i<128;i++)i2b(0xFF);
    i2p();ost(2,0,s,1);
}

/* draw sequence list screen */
static void oled_seq(void){
    uint8_t i,r;
    static const char TF[4]={'A','R','I','T'};
    oclr();otit("SEQ v6");
    for(i=0;i<MAXS;i++){
        r=i+1;
        och(0,r,'1'+i,0);
        if(i<sc){
            och(8,r,TF[seq[i].type],0);
            onum(16,r,seq[i].val,4);
            ost(42,r,seq[i].cw?"CW":"CC",0);
            onum(66,r,seq[i].rpm,2);
        }else{
            ost(8,r,"---",0);
        }
    }
    ost(0,7,"RUN",0);
    if(savedc) ost(60,7,"6 LOAD",0);
}

/* ======= MOTOR ======= */
static const uint8_t HS[8]={0x09,0x01,0x03,0x02,0x06,0x04,0x0C,0x08};
static volatile uint32_t vl=0;
static volatile uint8_t  vi=0,vcw=1;
static volatile uint16_t vr=0;

void __interrupt() isr(void){
    if(!PIR1bits.TMR1IF)return;
    PIR1bits.TMR1IF=0;
    TMR1H=(uint8_t)(vr>>8);TMR1L=(uint8_t)vr;
    if(vl){
        PORTD=(PORTD&0xF0)|HS[vi];
        vi=vcw?(vi+1)&7:(vi+7)&7;
        if(!--vl){PORTD&=0xF0;T1CONbits.TMR1ON=0;}
    }else{PORTD&=0xF0;T1CONbits.TMR1ON=0;}
}
static void mgo(uint32_t s,uint8_t cw,uint8_t rpm){
    uint32_t t=37500000UL/((uint32_t)SPR*rpm);
    if(t>65534)t=65534;
    vr=(uint16_t)(65536UL-t);vcw=cw;vl=s;
    TMR1H=(uint8_t)(vr>>8);TMR1L=(uint8_t)vr;
    T1CONbits.TMR1ON=1;
}
static void mstop(void){T1CONbits.TMR1ON=0;vl=0;PORTD&=0xF0;}

/* wait for motor/time — * = abort -> returns 1, normal -> returns 0 */
static uint8_t mwait(uint8_t secs){
    __delay_ms(150);
    if(!secs){
        while(vl){if(ks()=='*'){mstop();return 1;}}
        mstop();  delay_ms(200);return 0;
    }
    while(secs>0){
          delay_ms(500);if(ks()=='*'){mstop();return 1;}
        __delay_ms(500);if(ks()=='*'){mstop();return 1;}
        secs--;
    }
    mstop();__delay_ms(200);return 0;
```

```
}

/* intermittent: spin rs sec, idle is sec, cycs times. * = abort */
static uint8_t mwait_int(uint8_t rs,uint8_t is,uint8_t cycs,uint8_t cw,uint8_t rpm){
    uint8_t c,s;
    for(c=1;c<=cycs;c++){
        oclr_row(6);ost(0,6,"CYC",0);onum(14,6,c,2);ost(26,6,"/",0);onum(32,6,cycs,2);
        mgo(0xFFFFFFFF,cw,rpm);
        if(mwait(rs))return 1;
        if(is){
            s=is;
            while(s>0){
                __delay_ms(500);if(ks()=='*'){mstop();return 1;}
                __delay_ms(500);if(ks()=='*'){mstop();return 1;}
                s--;
            }
        }
    }
    return 0;
}

/* ======= KEYPAD ======= */
static const char KM[4][4]={
    {'1','2','3','A'},{'4','5','6','B'},
    {'7','8','9','C'},{'*','0','#','D'}
};
static void kinit(void){TRISB=0x0F;OPTION_REGbits.nRBPU=0;PORTB=0xF0;}
static char ks(void){
    uint8_t r,c;
    for(r=0;r<4;r++){
        PORTB=0xF0&~(0x10<<r);__delay_us(50);
        for(c=0;c<4;c++){
            if(!(PORTB&(1<<c))){__delay_ms(15);
                if(!(PORTB&(1<<c))){
                    while(!(PORTB&(1<<c)))
                    ;
                    __delay_ms(15);PORTB=0xF0;return KM[r][c];
                }
            }
        }
    }
    PORTB=0xF0;return 0;
}
static char kw(void){char k;do{k=ks();}while(!k);return k;}

/* ======= INPUT ======= */
static uint32_t gnm(uint32_t mx,uint8_t allow0){
    char buf[6];uint8_t n=0;char k;buf[0]=0;
    while(1){
        k=kw();
        if(k>='0'&&k<='9'&&n<5){buf[n++]=k;buf[n]=0;ost(72,7,buf,0);}
        else if(k=='*'){
            if(n){buf[--n]=0;ost(72+(n*6),7," ",0);}
            else return 0xFFFFFFFF;
        }
        else if(k=='#'){
            if(!n&&allow0)return 0;
            if(!n)continue;
            uint32_t v=0;uint8_t i;
            for(i=0;i<n;i++)v=v*10+(buf[i]-'0');
            if((!allow0&&!v)||v>mx){
                ost(72,7,"ERR",0);__delay_ms(400);
                ost(72,7,"    ",0);n=0;buf[0]=0;
```

```
            }else return v;
        }
    }
}
static uint8_t gdir(void){
    ost(0,7,"1CW 2CC",0);
    while(1){
        char k=kw();
        if(k=='1'){ost(0,7,"CW      ",0);return 1;}
        if(k=='2'){ost(0,7,"CCW     ",0);return 0;}
        if(k=='*')return 0xFF;
    }
}

/* ======= SEQUENCE OPS ======= */
static void cpstep(Step*d,Step*s){
    d->type=s->type;d->val=s->val;d->val2=s->val2;
    d->cyc=s->cyc;d->cw=s->cw;d->rpm=s->rpm;
}
static void autosave(void){
    uint8_t j;
    for(j=0;j<sc;j++)cpstep(&saved[j],&seq[j]);
    savedc=sc;
}
static void autoload(void){
    uint8_t j;
    for(j=0;j<savedc;j++)cpstep(&seq[j],&saved[j]);
    sc=savedc;
}

static void addstep(uint8_t type){
    uint32_t val=0,idle=0,rpm;uint8_t dir;
    static const char* TN[4]={"ANG","ROT","INT","TIM"};
    oclr_row(6);oclr_row(7);
    ost(0,6,TN[type],0);
    if(type==0){ost(0,7,"DEG:",0);val=gnm(3600,0);if(val==0xFFFFFFFF)return;}
    else if(type==1){ost(0,7,"ROT:",0);val=gnm(99,0);if(val==0xFFFFFFFF)return;}
    else if(type==2){
        ost(0,7,"SPN:",0);val=gnm(99,0);if(val==0xFFFFFFFF)return;
        ost(0,7,"IDL:",0);idle=gnm(99,1);if(idle==0xFFFFFFFF)return;
        ost(0,7,"CYC:",0);
        {uint32_t cy=gnm(99,0);if(cy==0xFFFFFFFF)return;seq[sc].cyc=(uint8_t)cy;}
    }
    else{ost(0,7,"SEC:",0);val=gnm(99,0);if(val==0xFFFFFFFF)return;}
    dir=gdir();if(dir==0xFF)return;
    ost(0,7,"RPM:",0);rpm=gnm(15,0);if(rpm==0xFFFFFFFF)return;
    seq[sc].type=type;seq[sc].val=val;seq[sc].val2=(uint8_t)idle;
    seq[sc].cw=dir;seq[sc].rpm=(uint8_t)rpm;sc++;
    oled_seq();
}

/* run one iteration of full sequence, returns 1 if * pressed */
static uint8_t run_once(void){
    uint8_t i;uint32_t steps;
    for(i=0;i<sc;i++){
        /* show step n/total on row1 */
        oclr_row(1);
        ost(0,1,"ST",0);onum(14,1,i+1,1);ost(20,1,"/",0);onum(26,1,sc,1);
        oclr_row(5);oclr_row(6);
        if(seq[i].type==0){
            steps=(seq[i].val*(uint32_t)SPR)/360;if(!steps)steps=1;
            mgo(steps,seq[i].cw,seq[i].rpm);
            if(mwait(0))return 1;
```

```
        }else if(seq[i].type==1){
            steps=seq[i].val*(uint32_t)SPR;
            mgo(steps,seq[i].cw,seq[i].rpm);
            if(mwait(0))return 1;
        }else if(seq[i].type==2){

if(mwait_int((uint8_t)seq[i].val,seq[i].val2,seq[i].cyc,seq[i].cw,seq[i].rpm))return 1;
        }else{
            mgo(0xFFFFFFFF,seq[i].cw,seq[i].rpm);
            if(mwait((uint8_t)seq[i].val))return 1;
        }
    }
    return 0;
}


/* ======= MAIN LOOP ======= */
void main(void){
    uint8_t state=0; /* 0=menu 1=getcount 2=running 3=done */
    uint32_t count=0,iter=0;
    uint8_t aborted=0;


    ADCON1=0x07;TRISA=0;TRISD=0;PORTD=0;TRISC=0xFF;
    TRISCbits.TRISC3=0;PORTCbits.RC3=1;
    TRISCbits.TRISC4=0;PORTCbits.RC4=1;
    TRISD&=0xF0;PORTD&=0xF0;
    T1CON=0x30;TMR1H=TMR1L=0;
    PIR1bits.TMR1IF=0;PIE1bits.TMR1IE=1;
    INTCONbits.PEIE=1;INTCONbits.GIE=1;
    kinit();__delay_ms(300);oinit();


    /* splash */
    otit("SEQ v6");
    ost(16,3,"STEPR",0);
    __delay_ms(1000);


    while(1){
        /* ---- MENU ---- */
        oled_seq();
        while(1){
            char k=kw();
            if(k=='1'&&sc<MAXS){addstep(0);oled_seq();}
            else if(k=='2'&&sc<MAXS){addstep(1);oled_seq();}
            else if(k=='3'&&sc<MAXS){addstep(2);oled_seq();}
            else if(k=='4'&&sc<MAXS){addstep(3);oled_seq();}
            else if(k=='*'){sc=0;savedc=0;oled_seq();}
            else if(k=='6'&&savedc){autoload();oled_seq();}
            else if(k=='#'&&sc>0){
                /* ask count */
                oclr_row(6);oclr_row(7);
                ost(0,6,"CNT:",0);
                count=gnm(99,0);
                if(count==0xFFFFFFFF) break; /* * pressed = cancel */
                /* go run */
                iter=0;aborted=0;
                oclr();otit("RUN");
                while(1){
                    iter++;
                    oclr_row(7);ost(0,7,"ITR",0);onum(14,7,iter,3);
                    aborted=run_once();
                    if(aborted)break;
                    if(iter>=count)break;
                    __delay_ms(300);
                }
```

```
                /* auto save */
                autosave();
                /* done screen */
                oclr();
                otit(aborted?"ABORT":"DONE");
                ost(0,3,aborted?"ABORT":"DONE",0);
                ost(0,5,"RUN  CLR",0);
                /* flush and wait */
                __delay_ms(300);while(ks());
                while(1){
                    char dk=kw();
                    if(dk=='#'){autoload();break;}  /* reload + back to menu */
                    if(dk=='*'){sc=0;savedc=0;break;} /* clear + back to menu */
                }
                break; /* break inner while -> back to top -> oled seq() -> menu */
            }
        }
    }
}
```



Conclusion

This project successfully implements a programmable stepper motor sequence controller on the resource-constrained PIC16F877A using interrupt-driven half-stepping, a software open-drain bit-bang I2C OLED driver, and a fully interactive 4x4 keypad menu system. Version 6 of the firmware refines the CYCLE step type into a fully automatic Intermittent (INT) mode with explicit spin seconds, idle seconds, and a fixed cycle count, eliminating the need for user intervention during execution. The simplified run mode accepts a repeat count (1–99) and auto-saves the sequence on completion. The Timer1 ISR-based motor engine decouples motion control from the application logic, enabling real-time keypad abort and live OLED updates without blocking. The entire firmware compiles within the 7680-word bootloader boundary using the free XC8 compiler at optimisation level 1, demonstrating effective use of an 8-bit microcontroller for a practical, multi-feature embedded control application.